Agenda:

1. Sum of all elements from L to R
2. Prefix Sum
3. Sum of even indexed elements
4. Special Index
5. Maximum subarray sum with length K
6. Find sum of all subarrays sums

Q → Given an integer array of size N & Q queries, for every query → find sum from L to R.

$$\begin{array}{cccccccc} & 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ A = & [-3 & 6 & 2 & 4 & -5 & 2 & 8] \end{array}$$

L	R	
1	3	→ 12
2	6	→ 11
0	4	→ 4

$1 <= N <= 10^5$

$1 <= Q <= 10^5$

Bruteforce → ∀ queries, iterate from L to R & get sum.

```
for i → 0 to (Q-1) {        // B[Q][2]
    L = B[i][0]        R = B[i][1]
    sum = 0
    for j → L to R {
        sum += A[j]
    }
    print (sum)
}
```

TC = $\underline{O(Q * N)}$ SC = $\underline{O(1)}$

$\sim 10^5 * 10^5 \rightarrow 10^{10}$ iterations

$\Rightarrow$ TLE

OVERS	1	2	3	4	5	6	7	8	9	10
SCORE	2	8	14	29	$\boxed{31}$	49	65	79	88	$\boxed{97}$

Scoreboard after every over.

Prefix sum

Runs scored in 7^{th} over $\rightarrow$ $65 - 49 = \underline{16}$

Runs scored from 6^{th} to 10^{th} over $\rightarrow$ $97 - 31 = \underline{66}$

Runs scored in 10^{th} over $\rightarrow$ $97 - 88 = \underline{9}$

Prefix Sum

$$P[i] = A[0] + A[1] + \dots + A[i]$$

$$P[3] = \underbrace{A[0] + A[1] + A[2]}_{P[2]} + A[3]$$

$$\boxed{\begin{aligned} &P[i] = P[i-1] + A[i], \quad i > 0 \\ &P[0] = A[0] \end{aligned}}$$

	0	1	2	3	4	5
A =	[10	32	6	12	20	1]
P =	[10	42	48	60	80	81]

$P[0] = A[0]$

```
for i → 1 to (N-1) {
    P[i] = P[i-1] + A[i]
}
```

$x + y - y = x$

$TC = \underline{O(N)}$

Sum L — R → $A[L] + A[L+1] + \dots + A[R]$

$+ (A[0] + A[1] + \dots A[L-1])$ → $P[R]$

$- (A[0] + A[1] + \dots A[L-1])$ → $P[L-1]$

$$\boxed{\text{Sum } L \text{ — } R = P[R] - P[L-1]}$$

```
for i → 0 to (Q-1) {        // B[Q][2]
    L = B[i][0]        R = B[i][1]
    if (L == 0)   sum = P[R]
    else   sum = P[R] - P[L-1]
    print (sum)                          TC = O(Q)
}
```

Overall → TC = O(N + Q)   SC = O(N) → P[]

O(1)

~~A[0] = A[0]~~

```
for i → 1 to (N-1) {
    A[i] = A[i-1] + A[i]
}
```

$$\begin{array}{ccccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ A = [& 10 & \cancel{32} & \cancel{6} & \cancel{12} & \cancel{20} & \cancel{1}] \end{array}$$

↳ [10 42 48 60 80 81]

Q → Given an integer array of size N & Q queries, for every query → find sum from L to R of only even index elements.

$$\begin{array}{cccccccc} & 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ A = [& -3 & 6 & 2 & 4 & -5 & 2 & 8] \end{array}$$

L	R	
1	3	→ 2
2	6	→ 5
0	4	→ -6

$1 <= N <= 10^5$

$1 <= Q <= 10^5$

```
P[0] = A[0]

for i → 1 to (N-1) {
    if (i % 2 == 0)     P[i] = P[i-1] + A[i]  ←
    else     P[i] = P[i-1]  ←
}
```

$$\begin{array}{ccccccc} & 0 & 1 & 2 & 3 & 4 & \\ A = [& 2 & 4 & 3 & 1 & 5 &] \\ P = [& 2 & 2 & 5 & 5 & 10 &] \end{array}$$

```
for i → 0 to (Q-1) {      // B[Q][2]
    L = B[i][0]        R = B[i][1]
    if (L == 0)   sum = P[R]
    else    sum = P[R] - P[L-1]
    print (sum)
}
```

Overall → $TC = O(N + Q)$ $SC = O(N) \rightarrow P[\,]$

use A[] for storing prefix sum → $\underline{O(1)}$

Q → Given an integer array, count the no. of special index.

→ Index after removing which, sum of even index elements = sum of odd index elements.

i	A = [4 (0)	3 (1)	2 (2)	7 (3)	6 (4)	-2 (5)]	S_e	S_o	
0	3	2	7	6	-2		8	8	✓
1	4	2	7	6	-2		9	8	
2	4	3	7	6	-2		9	9	✓
3	4	3	2	6	-2		4	9	
4	4	3	2	7	-2		4	10	
5	4	3	2	7	6		12	10	

Ans = $\underline{2}$

Bruteforce → $\forall i$, find S_e & S_o after removal of $A[i]$.

If $S_e == S_o$ increase answer by 1.

$TC = \underline{O(N^2)}$ $SC = \underline{O(1)}$

$$\begin{array}{cccccc} & 0 & 1 & 2 & 3 & 4 \\ A = [& 4 & 1 & \cancel{3} & 7 & 10\ ] \end{array}$$

7 10

$S_o = 1 + 10 = \underline{11}$

$$\begin{array}{ccccccccccc} & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ A = [& 2 & 3 & 1 & \cancel{4} & 0 & -1 & 2 & -2 & 10 & 8\ ] \end{array}$$

i

0 -1 2 -2 10 8

$S_o = \underline{15}$

$S_e = \underline{8}$

if i^{th} element is removed →

S_e = Sum of even index from 0 to $(i-1)$ + Sum of odd index from $(i+1)$ to $(N-1)$

$$\boxed{\begin{aligned} S_e &= P_e[i-1] + (P_o[N-1] - P[i]) \\ S_o &= P_o[i-1] + (P_e[N-1] - P_e[i]) \end{aligned}}$$

Sol → 1) calculate P_e & P_o arrays

2) $\forall i$, calculate S_e & S_o

3) if $(S_e == S_o)$ ans++

$TC = \underline{O(N)}$ $SC = \underline{O(N)}$

Q → Given an integer array, find max subarray sum, for subarray of len = K.

$$\begin{array}{cccccccc} & 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ A = [& -3 & 6 & 2 & 4 & -5 & 2 & 8 \;] \end{array} \qquad K = 4$$

9 7 3 9 (Ans)

Bruteforce → ∀ subarrays of len = K, find sum & store max as answer.

$$\begin{array}{cccccccc} & 0 & 1 & 2 & 3 & 4 & 5 & 6 \\ A = [& -3 & 6 & 2 & 4 & -5 & 2 & 8 \;] \end{array} \qquad K = 4$$

0   K-1   N-1

end → $[(K-1) \quad (N-1)] \rightarrow N \cancel{-1} - K \cancel{+1} + 1$

$= \underline{\underline{N-K+1}}$

$TC = \underline{O((N-K+1) * K)}$

$K = N \rightarrow (N-N+1) * N = \underline{\underline{N}}$

$K = 1 \rightarrow (N-1+1) * 1 = \underline{\underline{N}}$

$K = \frac{N}{2} \rightarrow \left(N - \frac{N}{2} + 1\right) * \frac{N}{2} = \frac{N}{2} * \frac{N}{2} = \frac{N^2}{4}$

$TC = O(N^2)$

Sliding window

$$
\begin{array}{ccccccccc}
 & & i-K & & & & i & & \\
 & & 0 & 1 & 2 & 3 & 4 & 5 & 6 \\
A & = & [-3 & \boxed{6} & 2 & 4 & -5 & \boxed{2} & 8]
\end{array}
\qquad K = 4
$$

sum = $\underline{13}$

```
sum = 0
for i → 0 to (K-1) {
    sum += A[i]
}

ans = sum
for i → K to (N-1) {
    sum += A[i] - A[i-K]
    ans = max (ans, sum)
}
return ans
```

TC = $\underline{O(N)}$ SC = $\underline{O(1)}$

Q → Given an integer array, find sum of all possible subarray sums.

<u>sum</u>

$$A = \begin{array}{ccc} 0 & 1 & 2 \\ \end{array}$$
$A = [3 \;\; 2 \;\; 5]$

3 → 3

3 2 → 5

3 2 5 → 10

2 → 2

2 5 → 7

5 → + 5

32 (Ans)

Brutefource → ∀subarrays calculate sum & add in the answer.

$TC = O(N^2 * N) \rightarrow O(N^3)$

Prefix sum → $TC = O(N^2)$

$SC = O(N) / O(1)$

$TC = O(N)$

Contribution Technique

$$Ans = \sum_{\forall i} \text{contribution of } A[i]$$

$A[i] * (\# \text{subarrays where } A[i] \text{ is present})$

HW

$$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array}$$
$A = [3 \;\; -2 \;\; 4 \;\; -1 \;\; 2 \;\; 6]$

start → [0 1] → 2

end → [1 5] → 5

$2 * 5 = 10$